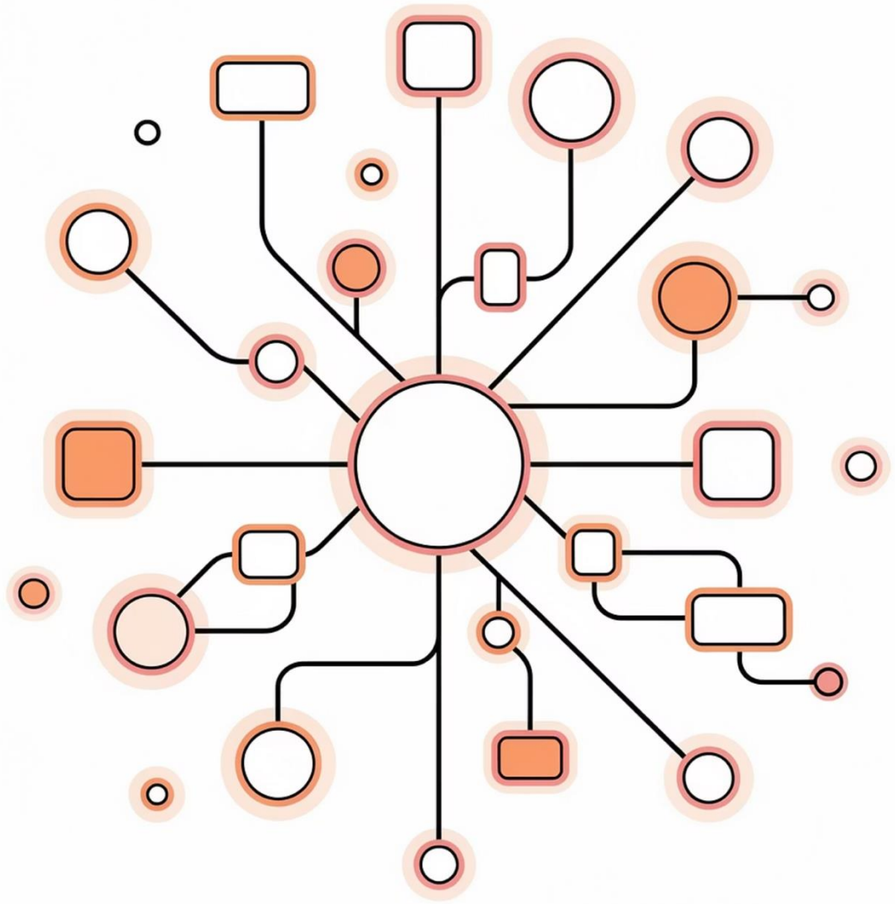


Database After Graduation

Week 4 — Transactions, Concurrency & Consistency

By **Narges Yarahmadi** · Feb 2026

İ MVNÕÑ ŎŅF ŎŌPÑŌPŒE

01

Transactions: The Atomic Unit of Work

ACID properties, Commit & Rollback

02

Race Conditions

The concurrency nightmare — lost updates, overselling

03

İ ØMŌŒEVNPÒŎŌ HŒĎÕVPÒŎŌ I ÑŌQÑÕŒE

Dirty Read, Non-Repeatable Read, Phantom Read

04

Tools of the Trade

Transactor Pattern, Concurrency Testing, Read Replicas

Why Concurrency Matters in Real Systems

In academic environments, database queries often execute sequentially. In production systems, this assumption immediately breaks.

The Banking Example

ÈÑÑÖÞÕPĘMÕMÕÑÑÈ **ÂĈĆĆ**
ĪÑØÞÑŒPÈ RÒÞOÑØMRŒÂĐĆ
ĪÑØÞÑŒPĘ RÒÞOÑØMRŒÂDĆ
ĘÖPOØÑMÑ `balance = 100`
ŒÒÖ ÞÕPMÕÑÖÞŒÊRB

Result: **Data inconsistency**
— the final stored balance is wrong.

Where Concurrency Happens

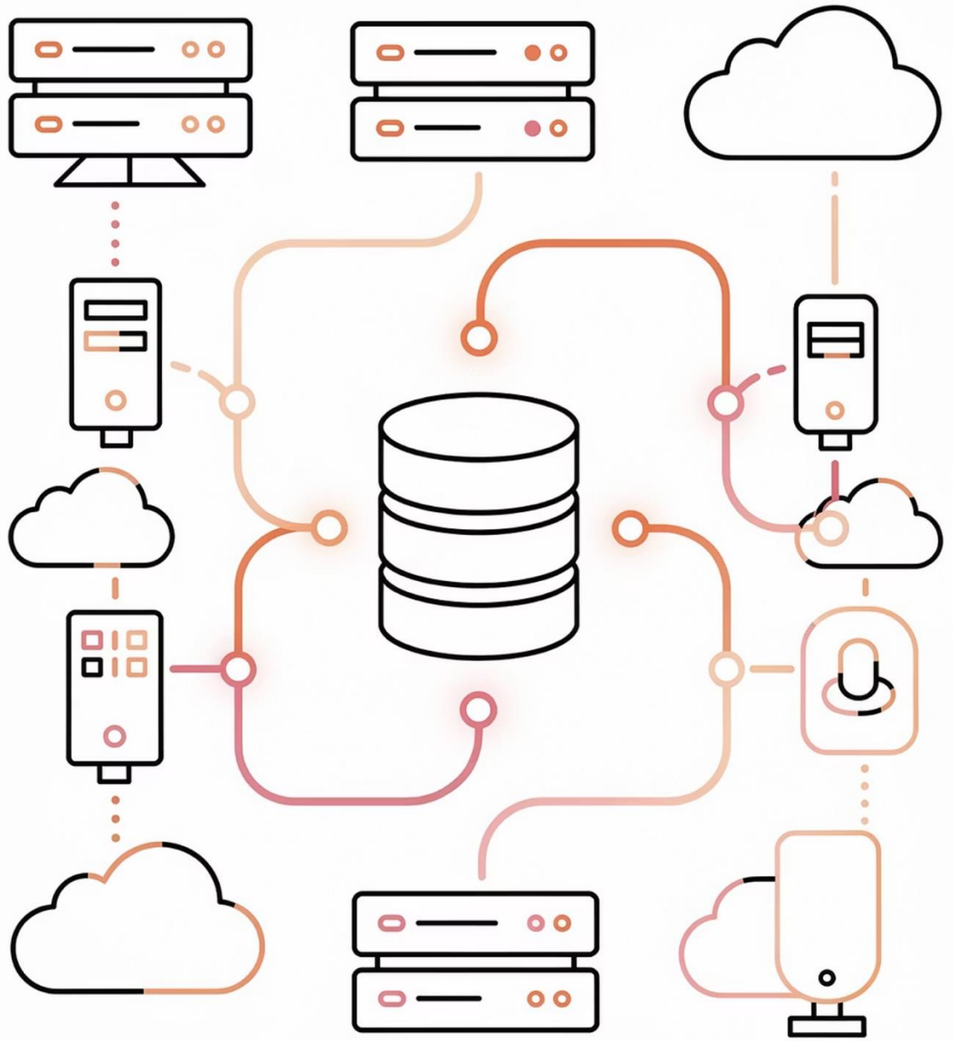
- Multiple users & API requests
- Microservices & background workers
- Scheduled jobs & distributed servers

Mobile app, web client, admin dashboard, and background billing jobs all hit the **same database simultaneously**.

Transactions

A transaction is a sequence of one or more operations (reads/writes) executed as a **single logical unit of work**. Transactions bring predictability to state changes.

The ACID Properties Define Reliable Transaction Processing



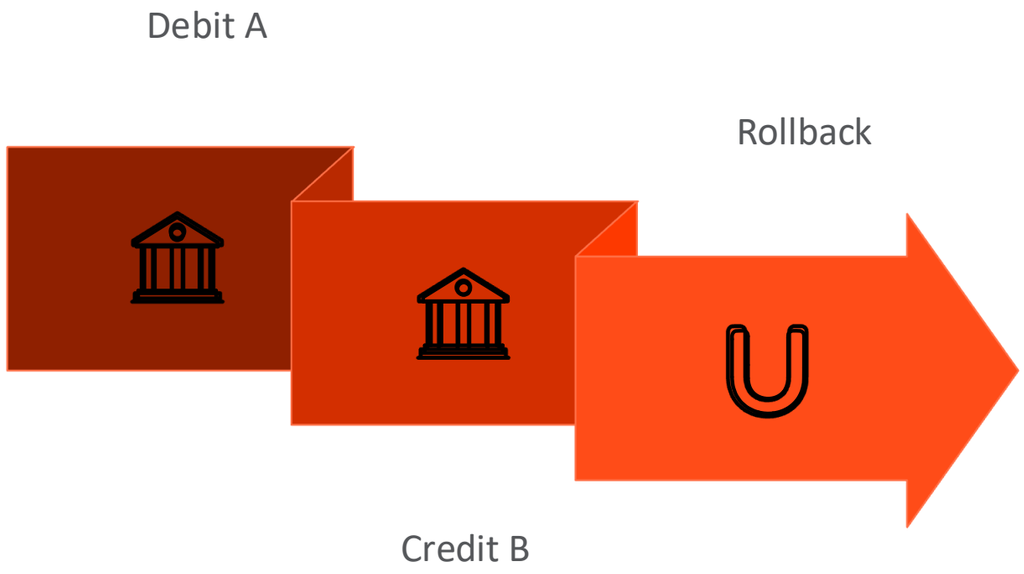
Ė PǑÖ ÒÚÒÞŘÈĖ ŐÕŎØÍ ŎPOÒŌN

If the system crashes mid-process, the transaction is rolled back.

All operations complete or none do. Example in Money transfer:

Debit Account A → Credit Account B

If credit fails → Debit must rollback.



F ŎŌOCÈÒDEÑŌŃŘÈIJ MÕÕŇ Ĭ PMPÑ PŎ IJ MÕÕŇ Ĭ PMPÑ

A transaction brings the database from one valid state to another, preserving defined rules (constraints, triggers).

Database moves from one valid state to another. Constraints remain preserved. Example:

Balance cannot be negative

Foreign keys remain valid

Isolation: Independent Execution

Concurrent execution of transactions results in a state that is equivalent to some serial (sequential) execution.

Transactions should not interfere with each other. Users should behave as if operations run independently.

GÞØMNÒÕÕDŘÈF ÖÖ Ö ÒPÑŇ ĞŎØŃQÑØ

Once a transaction is committed, it remains so, even in the event of a power loss or crash.

Once committed, Data survives crashes. Handled using:

Write-ahead logging

Disk persistence

Replication

The ACID Properties

Atomicity

Ė ŐŐŎŐŃŐØMPÒŎŎCĖŃŎŎ ŐŐŐNPÑ ŎØŐŎŎŐÑ ŃŎBHŅŃØŃŃŎÞ ŅMÒŎŎĖĖŃŃŃNÒÞ Ö Þ CĖP ØŎŎŐNMŃÔB

Consistency

Database moves from one valid state to another. Constraints (e.g., no negative balance) remain preserved.

Isolation

Transactions don't interfere with each other — equivalent to serial serial execution.

Durability

Once committed, data survives crashes via write-ahead logging, disk disk persistence, and replication.

Commit and Rollback: Save & Undo

İ OÑ İ OØÑÑ F ÖÖ Ö MŌÑŒ

BEGIN TRANSACTION

Marks the starting point of a transaction.

COMMIT

Saves all modifications permanently. Changes become visible to other transactions.

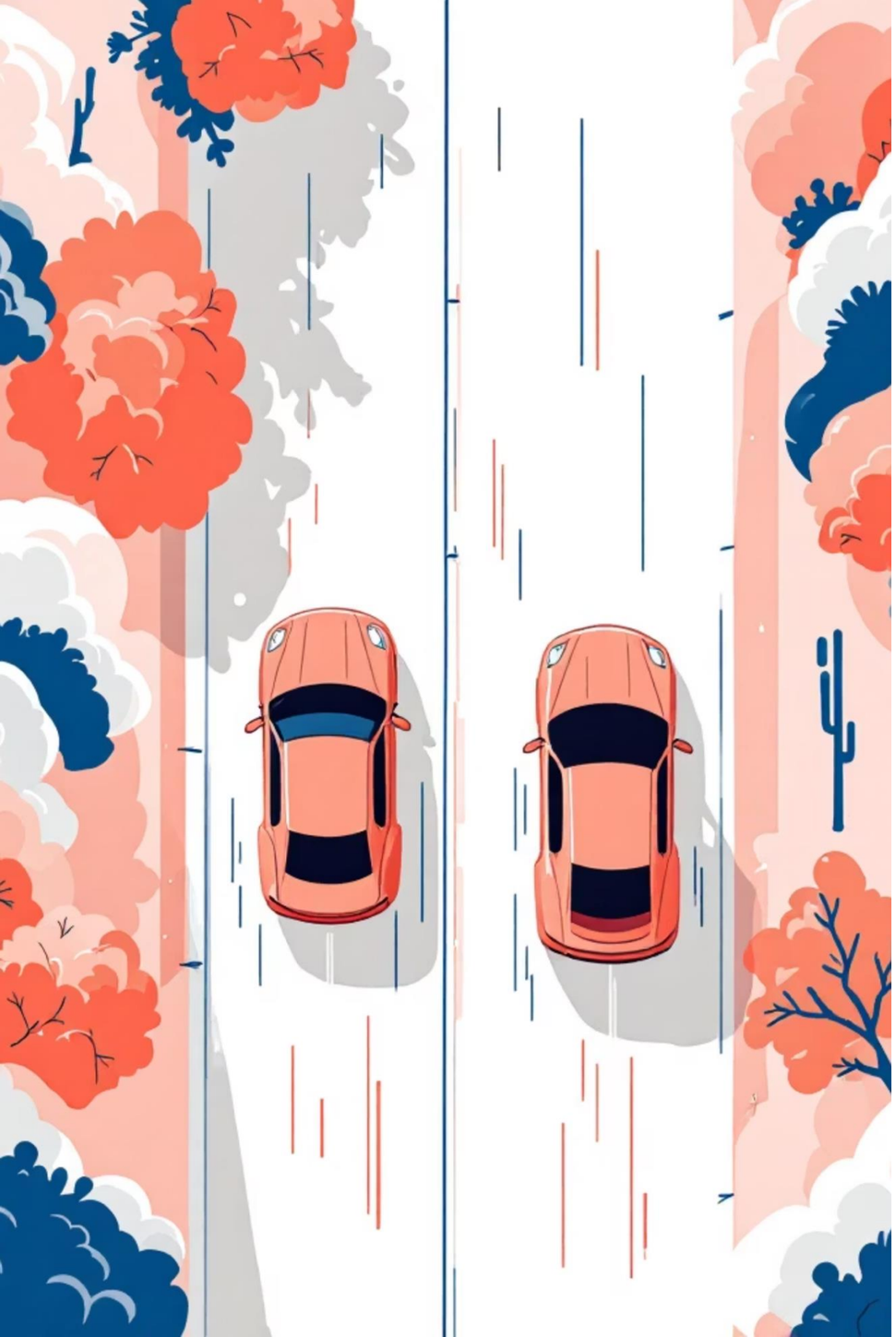
ROLLBACK

Aborts the transaction, undoing all modifications. Used when errors occur, validation fails, or business rules are violated.

Banking Transfer Example

```
BEGIN TRY
  BEGIN TRANSACTION TransferFunds;
  UPDATE Accounts
    SET balance = balance - 100
    WHERE id = 'A';
  UPDATE Accounts
    SET balance = balance + 100
    WHERE id = 'B';
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  ROLLBACK TRANSACTION;
  SELECT 'TRANSACTION FAILED';
END CATCH;
```

Takeaway: Never assume a multi-step operation succeeds until the COMMIT COMMIT is acknowledged.

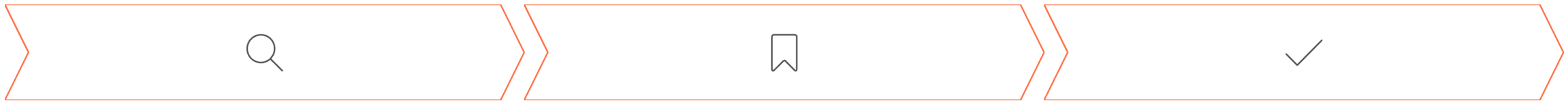


Race Conditions: The Concurrency Concurrency Nightmare

When multiple threads access shared data concurrently and the outcome outcome depends on execution timing, you have a **race condition** one of the most common production failures.

Example A: Ticket Booking

Two users purchase last seat.



Check available seats

Reserve seat

F ŎŌŅǾǾ ŐÞǾŃOMŒÑ

Both requests read availability **simultaneously**.

Result is **Overselling**.

⚠ İ OÑÎ ǾŌNǾÑÖ

Two users both see 1 seat available and both proceed to proceed to book it — resulting in a double booking of the of the same seat.

Example B: The "Lost Update" Problem

Bank Account Analogy

Imagine a bank account with **$1000**. Two concurrent requests arrive:

ĈBĪ ÑØÞÑŒPĖ ĂF ØÑÑÑÒÞÅ

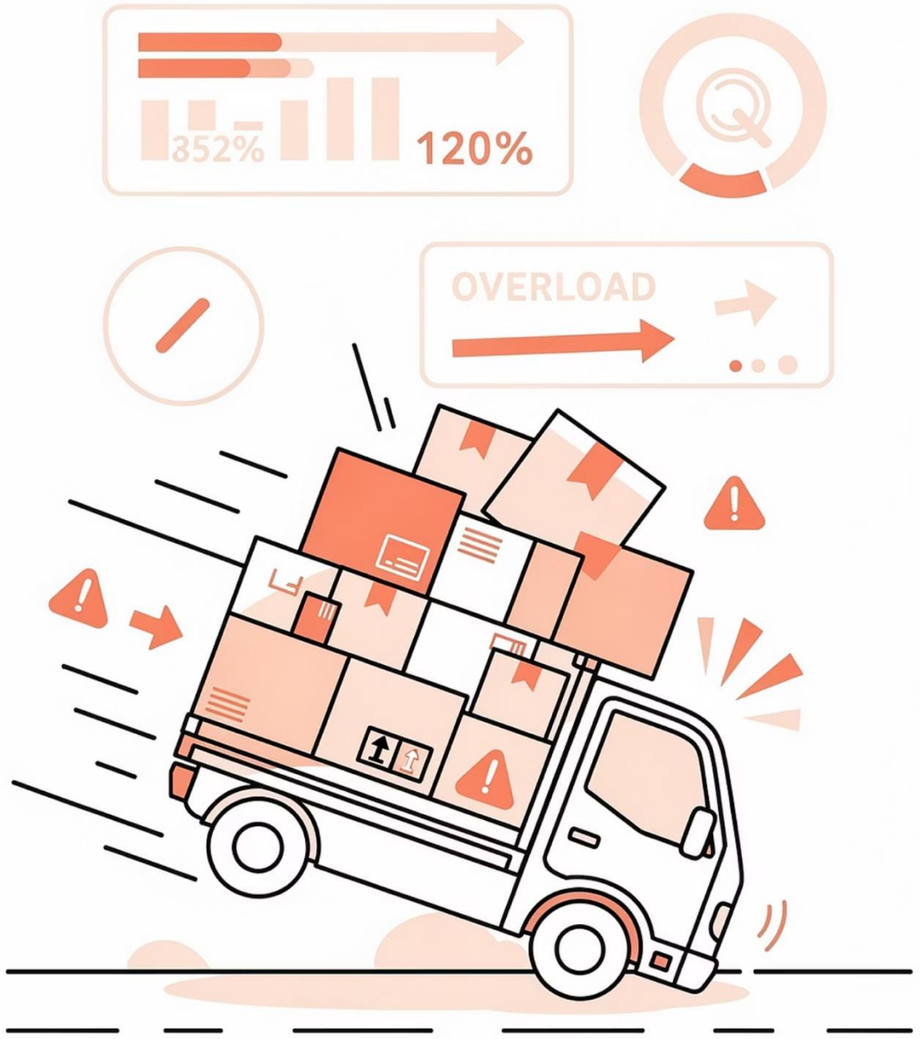
Add $100.

2. Request B (Debit)

Ï ÞNPØMÑP ÂDĆB

The Race:

Result: Despite two valid operations, the final balance is **$950**, not the correct **$1050**. One update was "lost".



Real-World Bug (False Mutex)

A common flaw is implementing a lock that only protects **part** of the system.

Consider a batch process (`executeBatch`) that uses a flag `isProcessingBatch` to prevent concurrent runs. However, if the standard API endpoint (`/assignParcel`) ignores this flag, a race condition occurs:

1. Batch reads capacity (0/100).
2. API reads capacity (0/100) before Batch commits.
3. Both assign parcels (50kg + 60kg).
4. Ī ÑCŒÞÕÞÈì ØÞ ÑÔ ÑMŐMÑÒÞŘ ÒŒEQÒŎÕMÞÑÑ ĂĈĈĆÔNJCĈĆĆÔNJÅB

Transaction Isolation Levels

To prevent race conditions without forcing all transactions to run one at a time (which kills performance), databases offer **Isolation Levels**. These levels define how transaction changes are visible to each other. They balance **consistency** against **concurrency**.

HÖ MŊÒŌÑ MŇMPMNMŒÑ OMŌŇŎŌŊ POŎÞ ŒMŌŇŒŎŅPÓMŌŒMÓPŎŎŌŒMP POÑ ŒMÖ Ñ PŎŌ ÑB

If the database forced this:

1. Transaction A finishes completely
2. then Transaction B starts
3. then Transaction C starts

✅ Ĩ ÑŐŅÑÑP ÑŎŐÑÑPŌÑŒE

❌ Terrible performance

Modern systems instead allow **transactions to run concurrently**, but they must control **how much transactions can see each other's changes**.

HŒŎMPŎŎ ŎÑQÑŒMÓÑ PŎÑÓÑŅÓÑÈ **Rules that define what one transaction is allowed to observe from another transaction.**

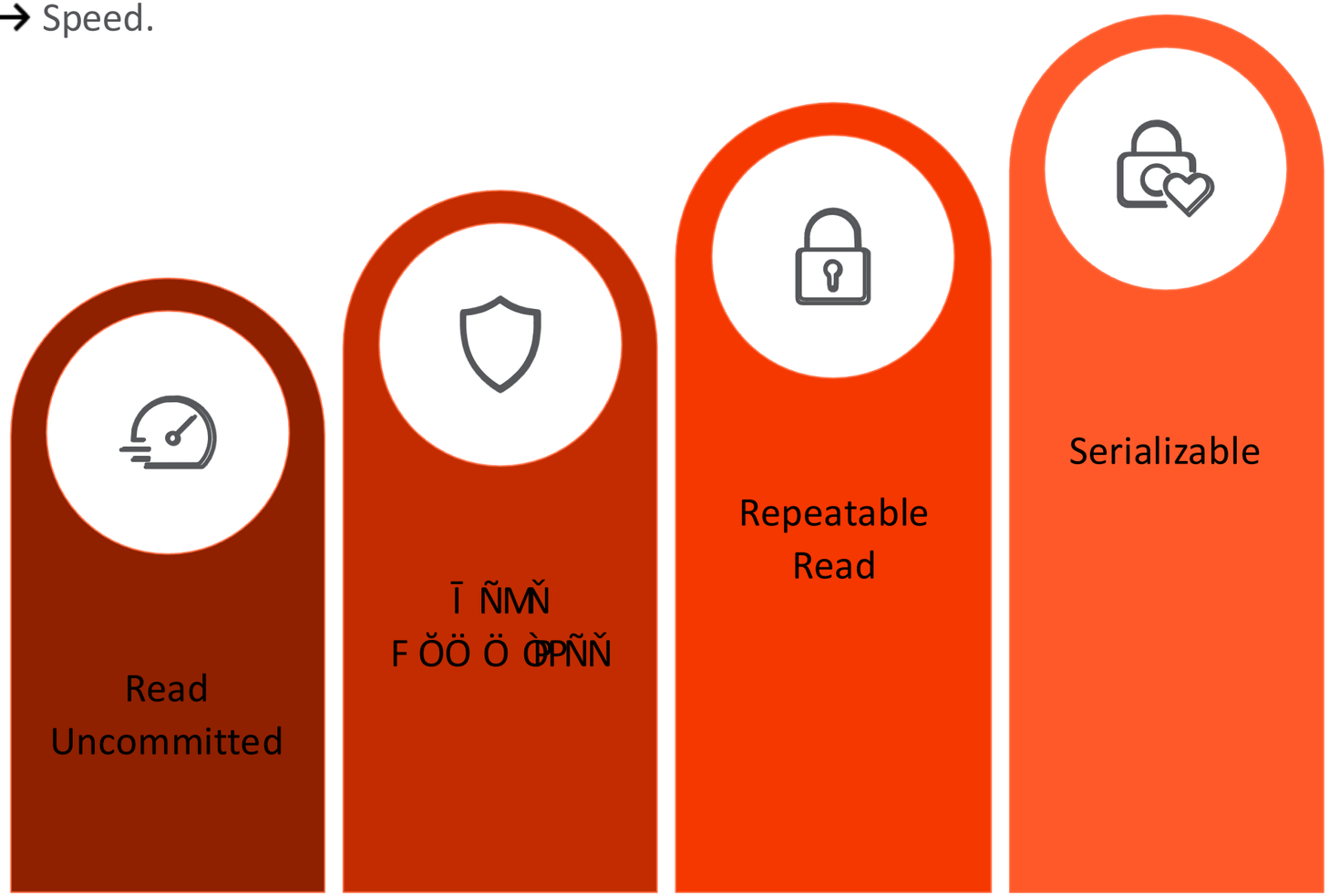
They balance:

Consistency ↔ Performance	Safety ↔ Speed

First, we define the **phenomena** we are trying to prevent:

Ì ØMŌCEMÑPÒŎŌ HŒĔÕMPÒŎŌ I ÑQÑÕŒE

To prevent race conditions without forcing serial execution (which kills performance), databases offer **Isolation Levels** — rules defining how much one transaction can observe from another. They balance Consistency ↔ Performance and Safety ↔ Speed.



First, we must understand the three phenomena these levels are designed to prevent: **Dirty Read**, **Non-Repeatable Read**, and **Phantom Read**.

Dirty Read

[illegible] **concurrent uncommitted transaction** [illegible]

The Scenario

Transaction A sets `balance = 0` but hasn't committed.

Transaction B reads `balance = 0`.

Transaction A crashes → `ROLLBACK`.

Actual balance returns to `500`.

Transaction B already used wrong data.

[illegible]

Financial systems **absolutely forbid** dirty reads. A payment approval based on based on a balance that was never real can cause catastrophic errors — funds — funds released that don't exist, fraud windows opened, and audit trails trails corrupted.

This is why **Read Uncommitted** isolation is rarely used in critical critical systems.

Non-Repeatable Read

Phantom Read

Non-Repeatable Read

Ī ÑMŇÒŊ PÕÑ **same row twice** ÒŌ ŎŌÑ PǾMŌŒMŃPÒŎŌ MŌŇ ŊÑPPÒŊ ŇÒŅŇǾŇŌP QMĐ ÑŒNÑŃMP ŒÑ MŌŎPOÑǾPǾMŌŒMŃPÒŎŌ Ö ŎŇÒŅŌŇ MŌŇ ŃŎÖ Ö ÒPPÑŇ ÒP ÒŌ NÑPR ÑÑŌB

Example A:

Transaction A reads `balance = 100`.

Transaction B updates it to `200` and commits.

Transaction A reads again: `200`.

Inside the **same transaction**.

Example B:

This breaks multi-step operations like:

calculate interest → verify funds → approve payment

where values change mid-process.

Ĩ OMŌPŎÖ Ī ÑMŇ

Running the same query twice and getting a **different set of rows** because another transaction inserted or deleted matching rows.

Transaction A queries orders `WHERE amount > 1000` → **5 rows**.

Transaction B inserts a $1500 order and commits.

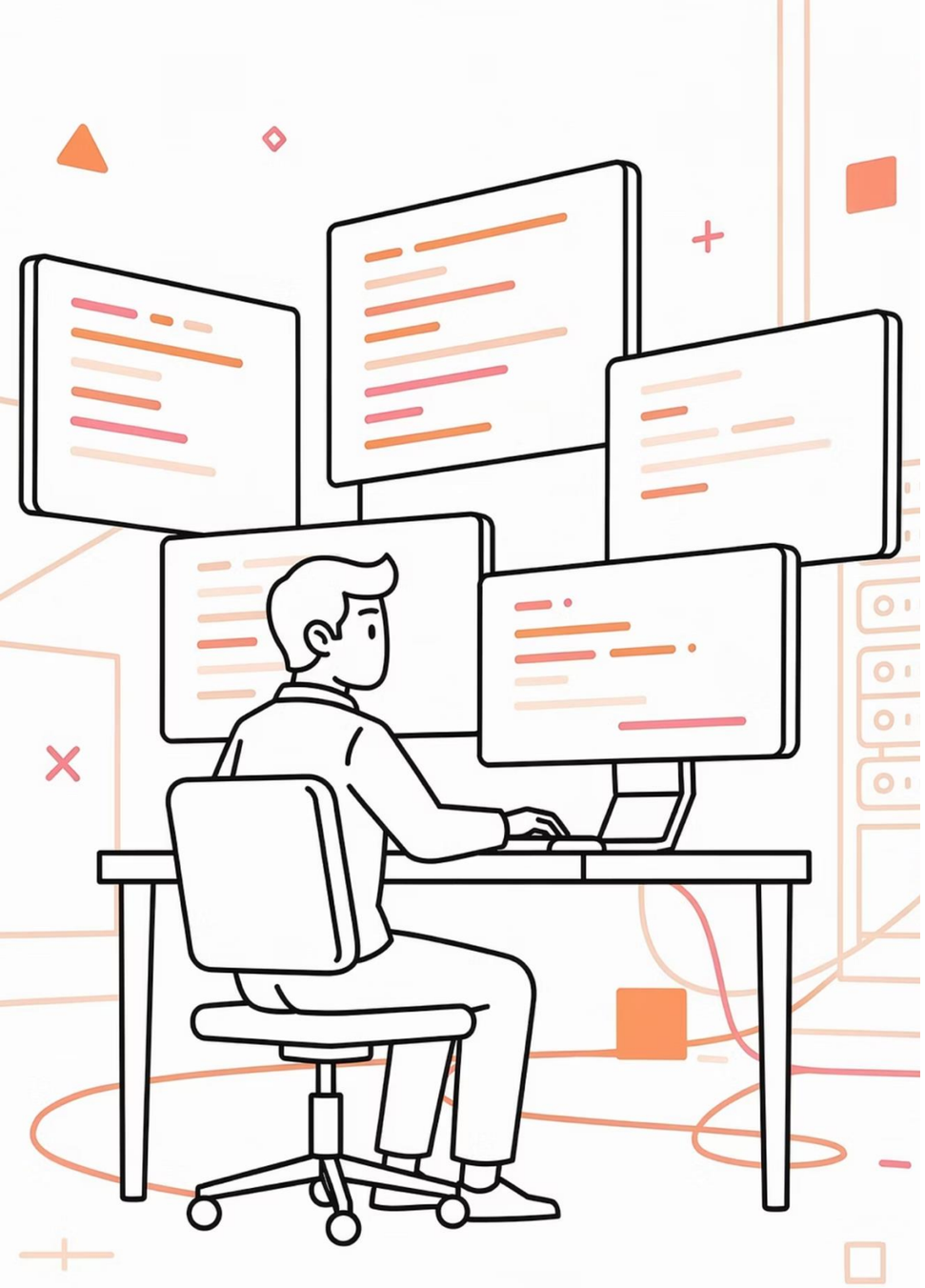
Transaction A re-runs the same query → **6 rows**. A new row appeared like a ghost.

Critical in: reporting, inventory checks, fraud detection, analytics.

Isolation Levels Comparison

Level	Dirty Read	Non-Repeatable	Phantom Read	Use Case & Analogy	Prevents	Performance
Read Uncommitted	Allowed	Allowed	Allowed	"The Gossip": Reading data instantly, even if it's not finalized. High risk, rarely used in critical financial systems.	❌ Dirty Reads ❌ Non-repeatable reads ❌ Phantom reads	Very High
Read Committed	Prevented	Allowed	Allowed	Default in PostgreSQL/Oracle. "The Official Statement." You only see data that has been officially committed. Prevents gossip, but data can change mid-transaction.	✅ Dirty Reads ❌ Non-repeatable reads ❌ Phantom reads	High
Repeatable Read	Prevented	Prevented	Allowed	Default in MySQL/InnoDB. "The Snapshot." If you read a row, you see the same value even if someone else changes it. However, new "phantom" rows can appear.	✅ Dirty Read ✅ Non-repeatable Read ❌ Phantom Reads (conceptually)	Medium
Serializable	Prevented	Prevented	Prevented	"The Vault." Transactions execute as if they were queued one after another. Highest consistency, lowest concurrency.	✅ Dirty Read ✅ Non-repeatable Read ✅ Phantom Reads (conceptually)	Lower

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```



TOOLS OF THE TRADE

Managing Concurrency in the Application Application Layer

G MPVNMDÑCEÓØÕQÒÑ PØMDBMPÒÕCEMDÑ ÒÐÕMPÒÕ ÕÑQÑÕBE Þ P ÒÕ Ö ÕÑÑØÕ BÉCBÑÖ CÆÖ ÕCP ÑÕÕÑÞ ØØÑÕÑÑR Ö ÒBEMDÑCEMÑPÞ MÕÕR OMDÓÑÕ ÒÕ POÑ **application layer**ÆÕÕP ÒÕCBÑÑ Ï Ï I B

1

API Server

Receives concurrent requests

2

Business Logic

Orchestrates operations

3

Repositories / ORM

Accesses the database

4

Database

ĜRÑÑÞ PÑCEPØMDBMPÒÕCE

Consistency must be controlled **above the database** as well. Transaction logic lives in your application code. application code.

Application-Level Transaction Management

Consider creating a task (like Jira/Trello). A single business operation requires multiple steps:

The Multi-Step Problem

1. Insert Task
2. [illegible]
3. Update Project Statistics
4. Log Activity

Why This Is Dangerous

If step 3 fails but steps 1 and 2 already succeeded, the database becomes becomes **inconsistent**:

- Task exists ✅
- Comments exist ✅
- Project counter incorrect ❌

This is why we need [illegible] Writing raw SQL transactions everywhere is error-prone — developers forget `commit()`, `rollback()`, and error handling.

The Repository & Transactor Pattern

Repositories (TaskRepository, CommentRepository, UserRepository) handle data access for their domain but operate independently — they don't coordinate transactions across operations.

The Transactor Pattern

A supervisory layer that controls the full transaction lifecycle: **BEGIN → execute business logic business logic → COMMIT or ROLLBACK**. If any step fails, it automatically triggers a rollback. a rollback.

When `repositoryTransactor.run(async () => { ... })` executes, a `BEGIN TRANSACTION` is implicitly started. All repository operations inside share that

The developer never needs to manually call `commit()` or `rollback()` — the Transactor handles it, eliminating human error.

Industrial Pseudo-code

```
class CreateTaskUseCase {
 constructor(
 private taskRepository: TaskRepository,
 private repositoryTransactor: RepositoryTransactor
 ) {}

 async handle(request) {
 return this.repositoryTransactor
 .run(async () => {
 const task = Task.create(
 request.taskName
 );
 await this.taskRepository
 .insert(task);
 // Auto-rollback if this fails
 return task;
 });
 }
}
```

Why Industry Favors the Transactor Pattern

Eliminates Human Error

No forgotten rollbacks, partial writes, or dangling data. The framework manages commits and rollbacks automatically.

Use Case–Level Boundaries

Transactions belong at the **Use Case level** (e.g., `CreateOrderUseCase`), not inside individual repository methods. Fragmenting transactions breaks atomicity.

Widely Adopted

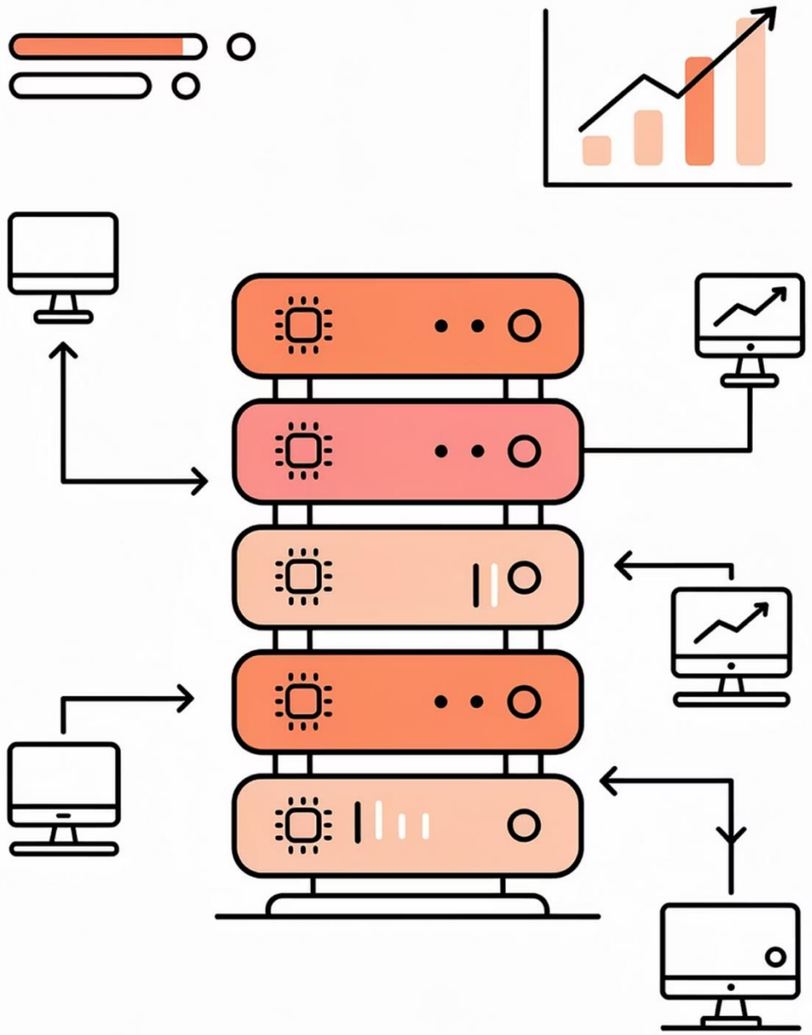
Standard in **NestJS, Spring Boot, Go-based based services**, and Domain-Driven Design Design systems. Framework responsibility, not responsibility, not individual developer concern.

J ÒÞOÖÞPÌ ØMŌCŒMÑPÖØ

Transaction safety = developer's full responsibility → high error risk in risk in complex workflows.

With Transactor

Transaction control = framework responsibility → developers focus on business logic.



API-Based Concurrency Testing

Race conditions rarely appear during development with single requests. In production, hundreds of users interact simultaneously. Engineers intentionally stress the application to expose hidden synchronization problems.

Apache JMeter

Industry standard, heavily used in banking and telecom. Simulates thousands of users, workflows, and authenticated sessions via a graphical interface.

ĦĎ

Modern, scriptable load testing tool written in JavaScript. Widely integrated into CI/CD pipelines.

CLI Benchmark Tools

Lightweight command-line tools to quickly hammer API endpoints without complex setup.

Interpreting Concurrency Test Results

Example Test Command

```
# 100 concurrent users, 1000 total requests
./benchmark -c 100 -n 1000 \
  -u http://localhost:3000/api/transfer \
  -m POST

# Output:
# Successful requests: 998
# Failed requests:       2

# Status distribution:
# 200: 998
# 409:   2  ← HTTP 409 Conflict
```

`-c 100` = 100 concurrent users

`-n 1000` = 1000 total requests

Ī ÑMŇÒŊ PON Ī ÑŒ ÕPCE

A beginner sees 998 successes and assumes good performance. An engineer **immediately investigates any failure**.

HTTP 409 Conflict may mean the database successfully prevented an unsafe an unsafe concurrent update via optimistic locking — a **positive outcome**. **outcome**.

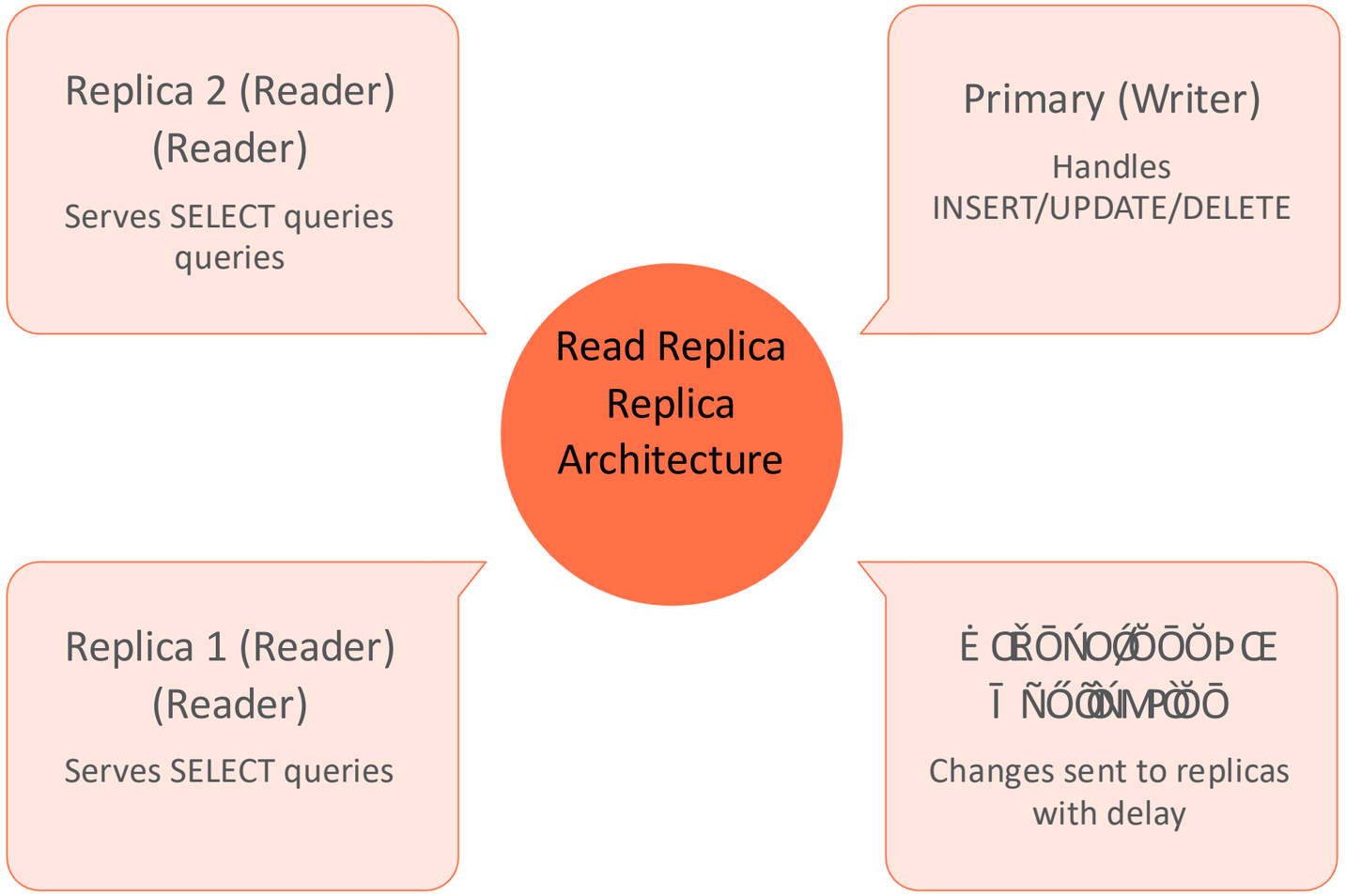
It can also signal deadlocks or logical race conditions requiring log analysis.

Industrial objective: Always prefer safe failure — a controlled controlled error is far better than a corrupted financial transfer. transfer.

Read Replicas & Consistency Trade-offs

As systems scale, the primary database gets overloaded with read queries. **Read Replicas** are copies that handle only `SELECT` queries — but they

— but they introduce a critical trade-off.



Ī ÑŐÕÓÑMPÕÕŌ ÒŒÞCŒMÕÕR **asynchronous** ŅŎØŐŃØŅŎØÖ MŌÓÑÑÆÖ ÑMŌÒŌŅ MØÑŐÕÓÑMÒŒEMÕR MŘCŒŒÕÓŅOPÕR NÑOÒŌÑ POÑŐØÖÖ MØŘ à POÒŒÒŒÉMÕÕÑ **replication lag**B

The Replication Lag Bug & Solutions

The Bug Scenario

1. User A posts a comment → goes to Primary

2. User A is redirected to the comment page

3. [illegible]

4. User A sees "Comment posted!" but the page shows no comments

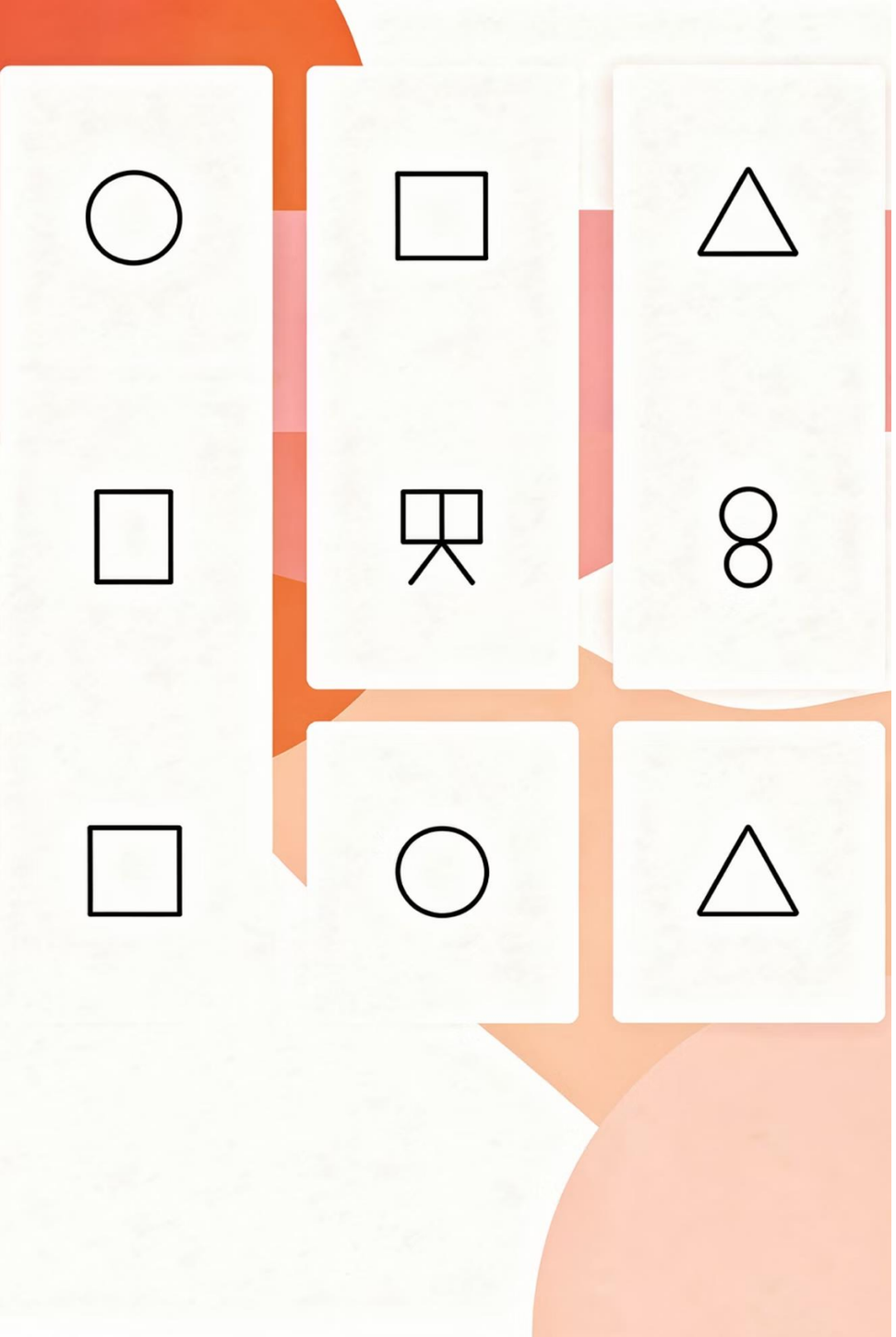
Solutions

Read-After-Write Consistency

Remember that User A just wrote something. For the next 5–10 seconds, route their reads to the **Primary**, not the Replica.

GTID Consistent Reads

Advanced setups (MySQL with with GTID) allow the replica to replica to wait until it has caught up to the specific transaction ID before returning returning data.



Ï ÞÖ Ö MØŐŘÈĦÑŘ İ MÔÑMR MŘCŒ

Transactions

COMMIT/ROLLBACK provide atomicity to prevent partial failures in multi-step operations.

Race Conditions

Concurrent operations interleaving incorrectly lead to lost updates, overselling, and data corruption.

Isolation Levels

Read Committed → Serializable: tools to balance performance against consistency guarantees.

Read Replicas

Ï ÑMÕÑ ØÑMÑ POØŐÞ ŊOŐÞ P NÞ P ÒŌPØŐÑÞ ÑÑ ÑQÑŌPÞ MŌÑÖŌCŒDEÑŌÑŘ à ØÑØÞ ØŒŌŊ MŐÓŌÑMPŒŌ ŌÆĐÑQÑŌŒŊÒÑPÖ OMŌÑŌÑ CEMŌÑ ØÑMÑCB

Next Step: Try the activity.pdf file — test your endpoints under concurrency and analyze the results firsthand.